RAMBOLL DEVELOPER PLATFORM

# Laying the Keel

A standards-first, self-service foundation for every new software project — from single sign-on to a compiling repository, living documentation, and a paved road, in minutes.

WHITEPAPER | PLATFORM ENGINEERING | Q4 TARGET

**KEEL — the project-initialization layer of the IDP**

**Scope** Project initialization: SSO hub, language blueprints, GitHub & Azure DevOps integration, GitHub Pages documentation

**Audience** Engineering leadership & the platform engineering team

**Status** Working draft for review · **Date** June 2026

# Executive summary

Why a project-initialization platform, and why now

Every Ramboll software project begins with the same invisible tax. Before a single line of business logic is written, a team spends days assembling the same scaffolding: a repository, a branching model, a CI pipeline, a documentation site, the security and compliance settings, and the tribal knowledge of "how we do things here." That work is repetitive, error-prone, and — critically — done slightly differently every time. The result is a sprawl of subtly inconsistent projects that are harder to operate, audit, and hand over.

This whitepaper proposes **Keel**: the project-initialization layer of the Ramboll Developer Platform (RDP). Keel is a self-service hub where an engineer signs in once through corporate single sign-on, chooses a blueprint for their language and project type, answers a short form, and — in minutes — receives a fully bootstrapped, standards-compliant project: a Git repository on GitHub or Azure DevOps, a `main` / `dev` / `staging` branching model with protection rules, a populated folder structure and README, an initial CI pipeline that already tests and compiles their code, and a published GitHub Pages documentation site that tells every contributor exactly where things go and how the team works.

Keel is not a portal that wraps existing tools with a thin UI. It is an opinionated *orchestration engine* that turns Ramboll's engineering standards into executable, version-controlled blueprints. We propose building it Rust-native — a single, statically-linked control-plane service with a small operational footprint, strong type guarantees around an inherently stateful and side-effecting workflow, and first-class libraries for the systems it must drive: `octocrab` for GitHub, Microsoft's `azure_devops_rust_api` for Azure DevOps, `openidconnect` for Entra ID single sign-on, and a Jinja-compatible template engine (`minijinja`) for rendering blueprints.

**~10 min**
Target time from sign-in to a compiling, documented, cataloged repository

**3**
MVP blueprints for Q4 — Python, Rust, TypeScript — GitHub-first

**1**
Golden path per language: one opinionated, supported, paved road

**100%**
Of new projects initialized through Keel are standards-compliant by construction

The strategic case is straightforward. Industry analysts place roughly 80% of large engineering organizations on a path to dedicated platform-engineering teams, and organizations that adopt internal developer platforms consistently report materially faster delivery and reduced operational toil. But the deeper value for Ramboll is *consistency at the point of creation*: standards that are applied automatically at initialization never have to be retrofitted, argued about in code review, or discovered during an incident. The cheapest place to enforce a standard is before the project exists.

We deliberately recommend keeping the Q4 MVP simple — three language blueprints, GitHub first, a clean initialization flow, and a documentation golden path — while architecting for a bolder horizon. The frontier this paper sketches includes AI-assisted scaffolding, blueprints that upgrade themselves into existing repositories as pull requests, continuous drift detection, and paved-road compliance scorecards. Those are the bets that turn a one-time scaffolding convenience into a living platform. The keel comes first; the rest of the ship is built upon it.

**What this paper is, and is not**

This is a vision-and-architecture whitepaper covering the *project-initialization layer* only — the foundation of the broader IDP. It is intentionally opinionated about technology to make the engineering tractable, but every recommendation is a proposal for discussion, not a final design. Deployment topology, exhaustive security review, and detailed cost modeling are deliberately out of scope and flagged for follow-up.

# Contents

# 1. The problem: initialization is where standards are won or lost

The cold-start tax, configuration drift, and the cost of "slightly different every time"

Ramboll is an engineering, architecture, and consultancy organization in which software is increasingly the medium of delivery — simulation tools, data pipelines, digital twins, client-facing applications, and internal automation. That software is written by a genuinely *polyglot* population: domain engineers reaching for Python to model a problem, data scientists in notebooks, and core software teams in TypeScript, .NET, and Rust. They sit in different business units, different countries, and different delivery contexts, many of them governed by client and regulatory obligations.

In that environment, the moment a project is created is the single highest-leverage point in its entire lifecycle — and today it is almost entirely unmanaged. Three concrete problems follow.

## 1.1 The cold-start tax

Starting a project "properly" is a surprising amount of undifferentiated work: create the repository, decide a folder layout, write a README nobody will finish, configure branches and protection rules, wire up a CI pipeline, add linting and formatting, set up a documentation site, connect secrets, and configure access. A team that does this carefully can lose days before writing any business logic. A team under deadline pressure skips most of it — and accrues debt that surfaces later as an audit finding or an incident. Either way the organization pays. Mature platform teams report that what was a multi-day setup collapses to a short form when initialization is automated: *fill out three questions and return to a fully scaffolded, compiling, cataloged project.*

## 1.2 Configuration drift and inconsistency

Because every team scaffolds by hand — or by copy-pasting last year's repository — no two projects are quite alike. Branch names differ (`master` vs `main`, no `staging`), CI does different things or nothing, documentation lives in five different places or in someone's head, and security settings are applied unevenly. This *configuration drift* is corrosive: it raises the cost of moving engineers between projects, defeats organization-wide tooling and reporting, complicates audits, and means a security or compliance fix has to be rediscovered and reapplied per repository rather than shipped once.

## 1.3 Standards that live in documents, not in code

Most organizations *have* engineering standards. They live in wiki pages and onboarding decks that are out of date the day they are written and that nobody reads at the only moment they matter — when creating the project. A standard that is not executable is a suggestion. The central premise of this paper is that the cheapest, most durable place to enforce a standard is **at initialization, by construction**, so that compliance is the default and divergence is the deliberate, visible exception.

> **The core insight**
>
> Quality, security, and consistency are far cheaper to *add at creation* than to *retrofit later*. A standard applied automatically when the repository is born never has to be argued in review, chased in an audit, or discovered in an outage. Keel exists to make the right way the easy way — and the default way.

## 1.4 What "good" looks like

The target state is a developer who wants to start a Python service signing in to one place, selecting "Python service," answering a few questions, and within minutes holding a repository that already compiles, already runs its tests in CI, already has branches and protections, and already publishes a documentation site explaining the project to the next person. The standards are present not because the developer is disciplined, but because the platform applied them. That is the outcome Keel is designed to deliver, and the rest of this paper describes how.

# 2. Vision: paved roads from the first commit

Golden paths, opinionated defaults, and self-service without losing control

Keel adopts the central idea of modern platform engineering: the **golden path** — an opinionated, well-documented, fully-supported way to build a given kind of software. A golden path is not a mandate and not a cage; it is the route that is so well-paved, so obviously easier, that teams choose it freely. The platform's job is to make that path the path of least resistance and to keep teams on it over time. Keel paves the first and most important stretch of that road: the creation of the project itself.

## 2.1 Design principles

Five principles shape every decision in this paper.

- **Opinionated by default, flexible at the edges.** A blueprint encodes one strong, sensible default for everything — layout, branches, CI, docs — so that 90% of teams never need to think about it. Escape hatches exist for the 10% with genuine reasons to differ, and those choices are explicit and recorded rather than silent.
- **Standards as code, not as prose.** Every standard Keel enforces is expressed in a version-controlled, reviewable, testable blueprint. Changing a standard is a pull request, not a memo.
- **Self-service, with control retained.** Developers serve themselves without raising tickets; the platform team retains control because the menu they choose from is the platform team's design. Autonomy for developers and governance for the organization are not in tension — the golden path delivers both.
- **Day-2 matters as much as day-0.** Initialization is the entry point, but a project lives for years. Keel is architected so that the standards it applies can be *kept current* in existing repositories, not just stamped once and abandoned (see §10 and §11).
- **Boring, observable, reversible.** The engine is a control plane that mutates real systems (Git, CI, identity). Every action is idempotent where possible, audited always, and reversible by design. Surprises are bugs.

## 2.2 The experience we are selling

To a developer, Keel should feel less like a tool and more like a concierge. They arrive at the Hub, already signed in through corporate identity. They see a small, curated catalog of project types — not an overwhelming wall of options, but the handful of paved roads Ramboll supports. They pick one, answer a short, friendly form (project name, owning team, target platform, a few language-specific switches), and press a button. A progress view shows the work happening — repository created, structure committed, branches protected, pipeline seeded, docs published, project catalogued — and ends with a tidy set of links: the repo, the live CI run, the documentation site. The first commit is already green.

## 2.3 Who benefits, and how

**VALUE BY STAKEHOLDER**

| Stakeholder | What Keel changes |
|---|---|
| **Developers & domain engineers** | Reclaim days otherwise lost to setup; start every project on a known-good foundation regardless of their depth of DevOps knowledge. |
| **Platform / DevOps team** | Encode their best practice once and have it applied everywhere; stop answering the same setup questions; shift from ticket-takers to road-pavers. |
| **Engineering leadership** | Achieve genuine consistency across a polyglot, distributed estate; faster time-to-delivery; a credible answer to "are our projects built to standard?" |
| **Security & compliance** | Controls applied uniformly at creation rather than retrofitted; a single place to update a control and have it propagate (see §10–11). |
| **New joiners** | Every project looks the same and documents itself the same way, so the second project is as legible as the first. Onboarding cost falls. |

> **A note on the name**
>
> In shipbuilding, *laying the keel* is the formal start of construction — the keel is the structural backbone every other part is fastened to, and the reference from which the whole vessel is measured. We use **Keel** as a working codename for the initialization layer for exactly that reason; the name is a proposal, not a requirement.

# 3. Where Keel fits: the IDP landscape and our position

What the market built, what we will reuse, and why we are building a focused layer

The internal developer platform (IDP) space is mature enough that we should borrow its vocabulary and avoid reinventing its concepts. It is also crowded enough that we should be deliberate about what we build versus adopt. This section places Keel on that map.

## 3.1 The established players and patterns

Three reference points matter. **Backstage**, created at Spotify, open-sourced in 2020 and donated to the CNCF, is the de-facto open framework for developer portals; its *Software Templates* (the "scaffolder") read a `template.yaml` manifest that defines input parameters and a sequence of actions, and publish the result to GitHub or Azure DevOps — exactly the initialization pattern Keel implements. **Port** and **Humanitec** represent the commercial portal and "platform orchestrator" categories: catalogs, blueprints, RBAC, and self-service actions, with golden paths defined by the platform team for developers to self-serve. Tools such as **OpsLevel** and **Cortex** popularized *scorecards* — scoring each service against reliability, security, and operational-readiness standards — which informs our governance thinking in §10.

The common pattern across all of them is stable and worth stating plainly: *a service catalog, plus paved roads (golden paths), plus automation, plus guardrails, plus production insight.* Keel is our implementation of the second and third of those — the paved-road and automation layer — with hooks designed to grow into the rest.

## 3.2 Build versus buy

Backstage is powerful but carries a well-known operational and maintenance burden (it is a TypeScript/Node application you must run, extend, and keep current). Commercial portals are faster to adopt but introduce per-seat cost, data-residency questions, and a ceiling on how deeply they integrate with Ramboll-specific workflow and compliance needs. For the *initialization layer specifically* — a focused, well-bounded problem — a purpose-built service is both tractable and strategically sensible: it is small, it encodes our exact standards, it has no licensing drag, and it can interoperate with a portal later rather than being replaced by one.

> **Our position**
>
> Keel is a **focused, opinionated initialization engine**, not a Backstage competitor. We build the paved-road automation ourselves because it is small and standards-specific; we explicitly preserve the option to surface Keel's catalog inside Backstage or a commercial portal later by emitting standard metadata (e.g. a `catalog-info.yaml`-style descriptor) for every project it creates.

## 3.3 Comparison at a glance

HOW KEEL RELATES TO COMMON OPTIONS (FOR THE INITIALIZATION PROBLEM)

| Dimension | Backstage (self-host) | Port / Humanitec | Keel (proposed) |
|---|---|---|---|
| **Primary scope** | Full portal + catalog + scaffolder | Portal / orchestrator (SaaS) | Initialization & golden-path engine |
| **Effort to adopt** | High — run & extend a Node app | Low–medium — configure SaaS | Medium — build a small service |
| **Fit to Ramboll standards** | High (you code it) | Medium (configurable) | Highest (purpose-built) |
| **Cost model** | Engineering time | Per-seat licensing | Engineering time, small footprint |
| **Lock-in** | Low | Medium–high | Low (we own it) |
| **Interop path** | — | — | Emits portable catalog metadata; can sit behind a portal later |

In short: we are not betting against portals. We are building the one piece that is most valuable to own outright — the engine that turns our standards into projects — and keeping every door open to integrate with a broader portal when the organization is ready.

# 4. Reference architecture

Three planes, one Rust-native control plane, and the systems it drives

Keel is organized into three planes — a model borrowed from platform engineering and adapted to the initialization problem. The **experience plane** is what developers touch; the **orchestration plane** is the Keel engine that does the work; and the **integration plane** is the set of external systems Keel drives on the developer's behalf. Keeping these separate means the UI, the engine, and the integrations each evolve independently.

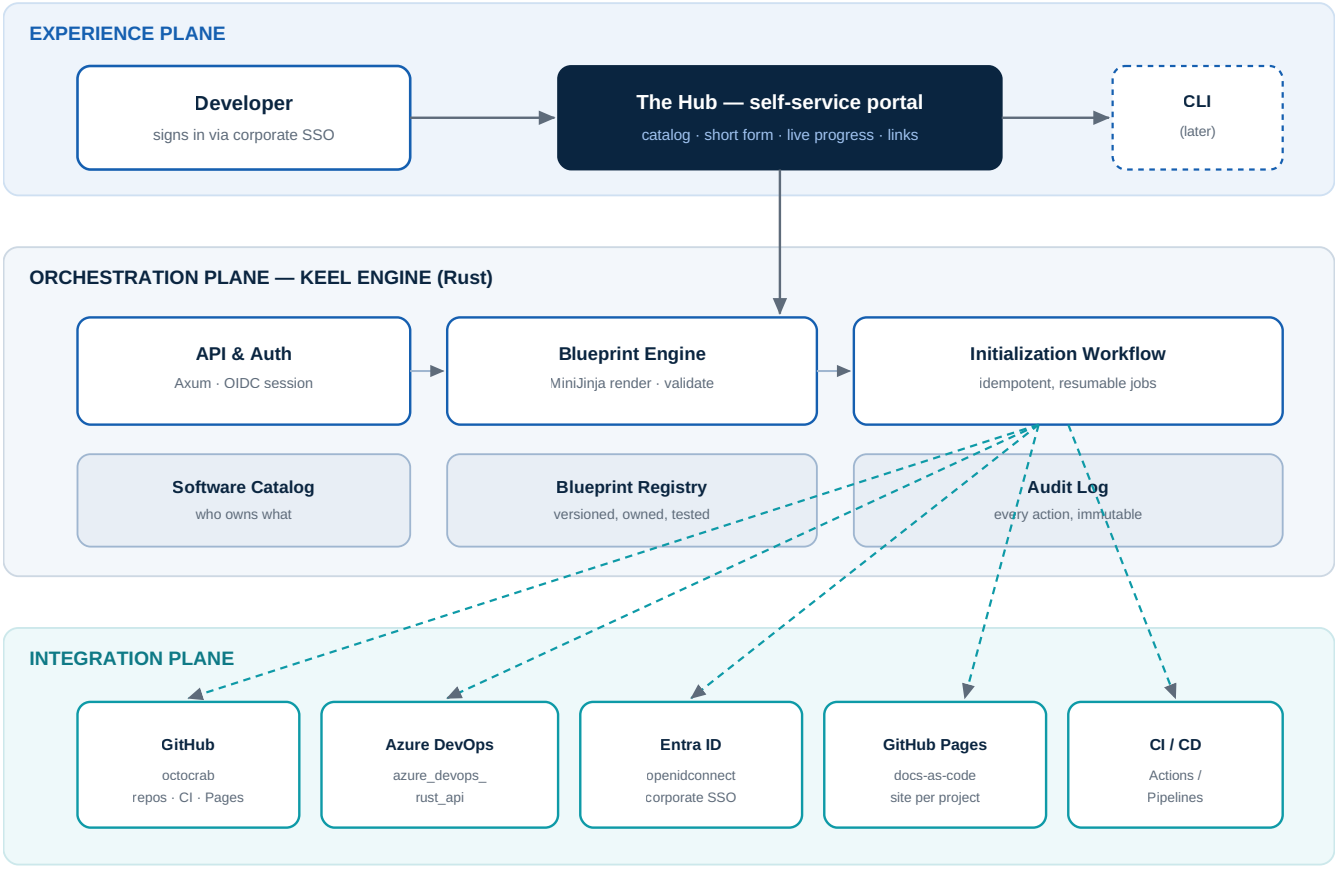


**Figure 1 — Keel reference architecture.** The experience plane (Hub) collects intent; the Rust orchestration plane renders blueprints and runs an idempotent initialization workflow; the integration plane drives GitHub, Azure DevOps, Entra ID, GitHub Pages, and CI. Dashed lines are side-effecting calls to external systems.

## 4.1 Why Rust for the control plane

Keel is a control-plane service: long-lived, concurrent, security-sensitive, and defined by side effects on critical systems. Rust is an unusually good fit for that profile, and standardizing on it aligns with Ramboll's stated direction to build in Rust where practical.

- **One small, static artifact.** Keel compiles to a single statically-linked binary with no runtime to install — trivial to containerize, cheap to run, and easy to operate. The operational footprint of the platform should not itself become a platform problem.
- **Correctness for a side-effecting workflow.** Initialization is a sequence of irreversible-ish operations against external systems. Rust's type system and exhaustive error handling (`Result`, the `?` operator, enums for state) make the "what can go wrong and what do we do" surface explicit at compile time rather than discovered in production.

- **First-class libraries for exactly our targets.** The integration plane is well-served by mature crates: `octocrab` (GitHub), Microsoft's own `azure_devops_rust_api` (Azure DevOps REST 7.1), `openidconnect` / `axum-oidc` (Entra ID), and `axum` for the web layer. We are not building on exotic foundations.
- **Fearless concurrency.** Initializing several projects at once, each fanning out to multiple APIs, is naturally concurrent. Rust's async model ( `tokio` ) handles this safely without the data-race class of bugs.

The honest trade-off is talent: Rust expertise is rarer than Node or Python, and the initial build is more deliberate. We address this in §13. The MVP is deliberately small precisely so that this trade-off is bounded.

## 4.2 Component responsibilities

ORCHESTRATION-PLANE COMPONENTS

| Component | Responsibility |
|---|---|
| **API & Auth (Axum)** | HTTP API and server-rendered/SPA Hub; terminates the OIDC session; maps Entra groups to roles; rate-limits and authorizes every request. |
| **Blueprint Engine** | Loads a blueprint, validates the developer's parameters against its schema, and renders the template tree with MiniJinja into an in-memory file set. |
| **Initialization Workflow** | Executes the ordered, idempotent steps (create repo → commit → branches → protection → CI → docs → catalog). Resumable and reversible; emits progress events to the Hub. |
| **Blueprint Registry** | Stores versioned, owned, tested blueprints. The source of truth for "what a Ramboll project looks like." |
| **Software Catalog** | Records every project Keel creates — owner, language, repo, docs URL — and emits portable metadata for downstream portals. |
| **Audit Log** | Append-only record of who initialized what, with which blueprint version, and the result of every external call. |

# 5. Anatomy of a blueprint

The unit of standardization: a manifest, a template tree, and post-actions

The blueprint is the heart of Keel. It is the executable, version-controlled embodiment of "how Ramboll builds a *this kind of* project." A blueprint has three parts: a **manifest** that declares the questions to ask and the work to do, a **template tree** of the files to render, and a set of **post-actions** that configure the surrounding systems (branches, CI, docs, catalog).

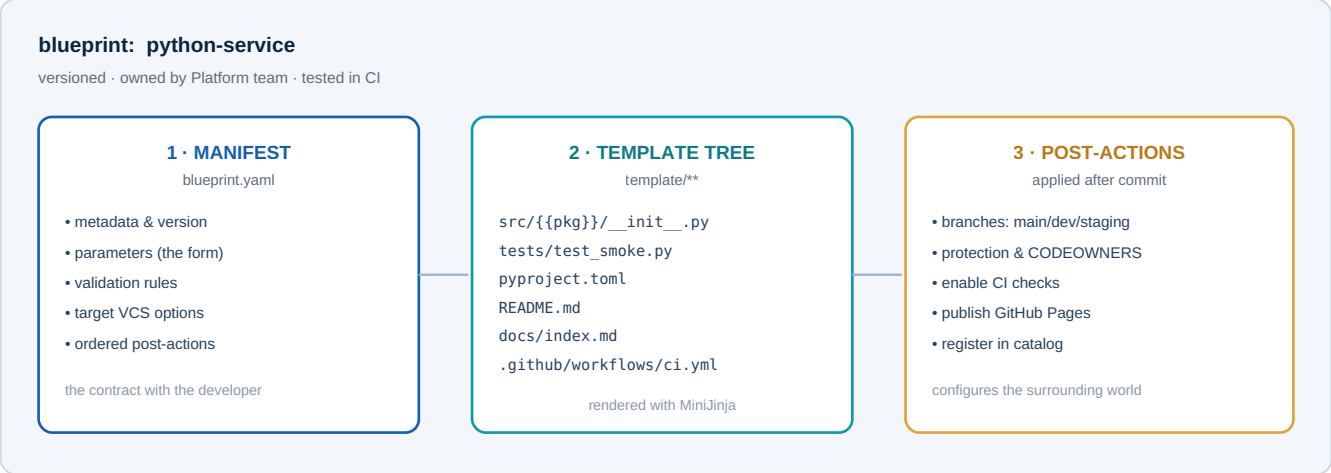


**Figure 2 — Blueprint structure.** One blueprint = a manifest (what to ask, what to do), a template tree (the files), and post-actions (how to configure GitHub/Azure DevOps, CI, docs, and the catalog). Blueprints are themselves stored in Git, reviewed, versioned, and tested.

## 5.1 The manifest: the contract with the developer

The manifest declares the parameters the Hub turns into a form and the post-actions the workflow executes. Modeling it on the widely-understood Backstage `template.yaml` convention keeps it legible to anyone who has used an IDP and preserves our interop story. A minimal example:

```yaml
# blueprint.yaml — Python service (excerpt)
apiVersion: keel/v1
kind: Blueprint
metadata:
  name: python-service
  title: "Python Service"
  version: "1.4.0"
  owner: platform-engineering
parameters:
  - id: project_name
    type: string
    pattern: "^[a-z][a-z0-9-]{2,40}$"
    required: true
  - id: owning_team
    type: enum
    source: entra-groups        # populated from the user's groups
  - id: target
    type: enum
    values: [github, azure-devops]
    default: github
postActions:
  - create_repository
  - commit_template
  - setup_branches      # main, dev, staging + protection
```

```
  - enable_ci
  - publish_docs          # GitHub Pages
  - register_in_catalog
```

The manifest is data, not code: the Hub renders the form from `parameters`; the workflow runs the named `postActions` in order. Adding a question or a step is a reviewable change to one file.

## 5.2 Rendering: MiniJinja over the template tree

Files under `template/` are rendered with **MiniJinja**, a minimal-dependency, Jinja2-compatible engine for Rust. Jinja2 syntax is familiar to the Python-heavy population Keel serves, and MiniJinja's single-dependency footprint suits a control-plane binary. Both file *contents* and file *paths* are templated, so `src/{{ project_name }}/__init__.py` becomes a correctly-named module. (Tera is a viable alternative with a similar feature set; MiniJinja is preferred for its smaller dependency surface.)

## 5.3 The MVP blueprint catalog

We propose three blueprints for the Q4 MVP — covering the bulk of new work — and name the obvious fast-followers. Each ships a project that *compiles and passes a smoke test on the first CI run*; that "green from birth" property is a deliberate quality bar.

PROPOSED BLUEPRINT CATALOG

| Blueprint | Phase | What it ships |
|---|---|---|
| **Python service** | MVP | `pyproject.toml`, `src/` layout, `pytest` smoke test, `ruff` + `black`, typed with `mypy`, CI that lints + tests, MkDocs docs site. |
| **Rust service** | MVP | Cargo workspace, `clippy` + `rustfmt`, unit + integration test skeleton, CI that builds + tests + lints, mdBook docs site. |
| **TypeScript service** | MVP | Node/TS project, ESLint + Prettier, `vitest` smoke test, build + typecheck CI, MkDocs docs site. |
| .NET service | FAST-FOLLOW | Solution + project layout, analyzers, xUnit tests, CI build/test — common in Ramboll's Microsoft-aligned estate. |
| Data / notebook project | FAST-FOLLOW | Reproducible environment, data-handling and structure conventions for analysts and data scientists. |

## 5.4 Blueprints are software

A blueprint is treated like any other product: it lives in Git, is owned by the platform team, is reviewed via pull request, and is *tested in CI* — every blueprint is initialized into a throwaway repository on each change to prove it still produces a green project. This is what lets us evolve standards confidently: a change to the golden path is validated before it can affect a single real team.

# 6. The initialization flow, end to end

From sign-in to a green, documented, cataloged repository

The initialization workflow is the orchestration plane's signature operation: an ordered, idempotent sequence that transforms a developer's short form into a fully configured project. The steps are the same regardless of language; the blueprint supplies the content and the target supplies the API.

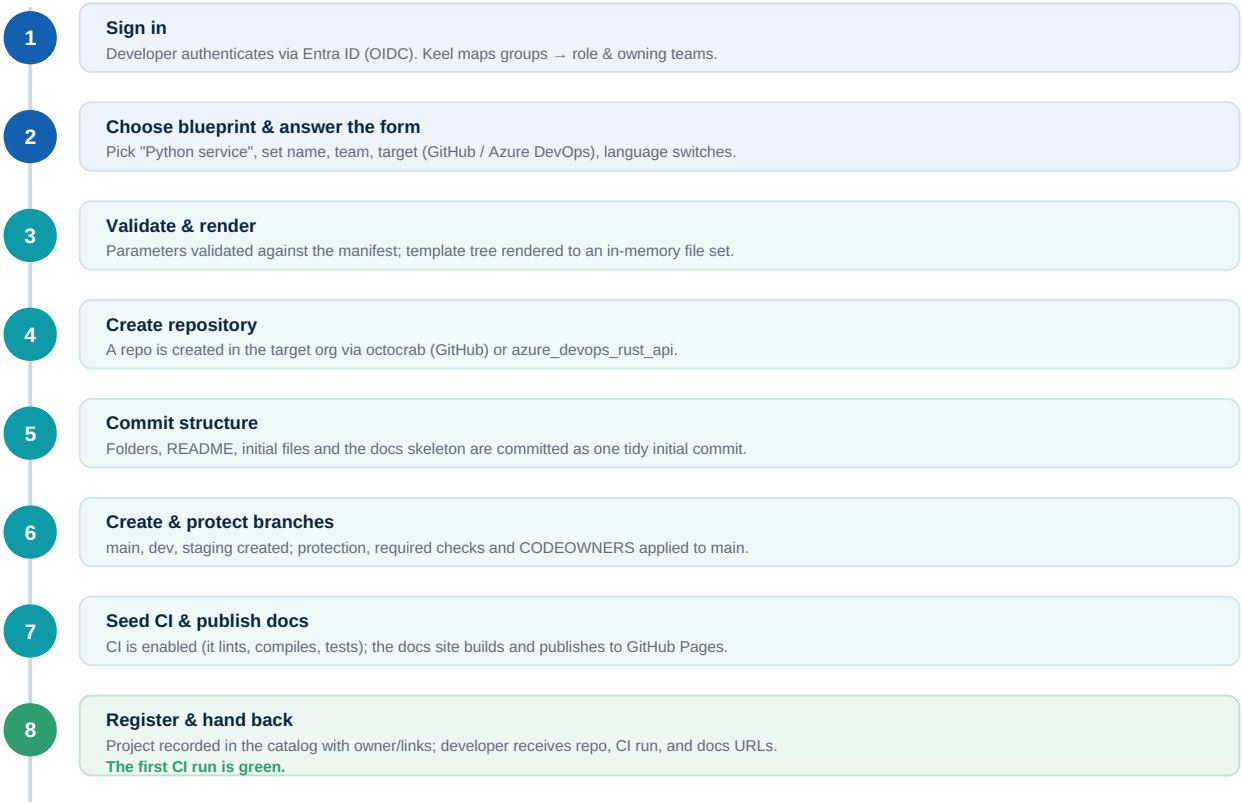


**Figure 3 — The initialization workflow.** Eight ordered, idempotent steps. Each emits a progress event to the Hub so the developer watches the project being built. Any step that fails is retried or cleanly rolled back; nothing is left half-created.

## 6.1 Idempotency, rollback, and audit

Because the workflow mutates external systems, three properties are designed in. **Idempotency:** each step checks current state before acting, so a retried run does not create a second repository or duplicate branches. **Rollback:** if a later step fails irrecoverably, earlier effects are unwound (or the partial project is quarantined and flagged) rather than left in an ambiguous state. **Audit:** every step writes who/what/when/result to the append-only log, including the exact blueprint version used — so any project can be traced back to the standard it was born from.

## 6.2 The repository bootstrap, in Rust

The core of steps 4–6 against GitHub uses `octocrab`. Keel authenticates as a GitHub App, scopes down to the org installation for a short-lived token, then creates the repo, commits the rendered files, cuts the branches, and applies protection. The snippet below is illustrative but faithful to octocrab's documented API:

```
// Authenticate as the Keel GitHub App, then scope to the org installation
let key = EncodingKey::from_rsa_pem(app_pem.as_bytes())?;
```

```rust
let app = Octocrab::builder().app(app_id, key).build()?;
let gh  = app.installation(installation_id)?;   // short-lived token

// 4. Create the repository in the target org
let repo = gh.post(format!("/orgs/{org}/repos"), Some(&spec)).await?;

// 5. Commit the rendered blueprint (Git Data API for a single clean commit)
for file in rendered_files {
    gh.repos(&org, &name)
      .create_file(&file.path, "chore: scaffold from blueprint", &file.body)
      .branch("main")
      .send().await?;        // simplified; batched via blobs+tree in practice
}

// 6. Cut dev + staging from main, then protect main
let main = gh.repos(&org, &name)
    .get_ref(&Reference::Branch("main".into())).await?;
for br in ["dev", "staging"] {
    gh.repos(&org, &name)
      .create_ref(&Reference::Branch(br.into()), main_sha).await?;
}
gh.put(format!("/repos/{org}/{name}/branches/main/protection"),
       Some(&policy)).await?;   // HTTP escape hatch for partial-typed APIs
```

Two honest notes. (1) For a single tidy initial commit rather than one-commit-per-file, the Git Data API (create blobs → build a tree → create a commit → update the ref) is used; `create_file` is shown for brevity. (2) octocrab's strongly-typed surface does not cover every endpoint (org repo creation, branch protection), so Keel uses octocrab's documented lower-level `post`/`put` HTTP methods for those — same auth, same client. The Azure DevOps path is structurally identical using `azure_devops_rust_api` (Git + Build + Core areas).

# 7. Repository standards: structure, branching, and CI

What every Keel-born repository looks like on day one

This section defines the concrete artifacts a blueprint produces. These are the visible, opinionated standards developers experience — and the ones that, applied uniformly, make the estate legible.

## 7.1 Folder structure and initial files

Every repository ships a predictable skeleton so that a developer moving between projects always knows where things live. Language specifics vary, but the top level is consistent:

STANDARD TOP-LEVEL FILES AND FOLDERS

| Path | Purpose |
|---|---|
| `README.md` | Generated, not blank: what the project is, how to run it, where the docs are, who owns it. The front door. |
| `docs/` | Docs-as-code source (see §8). Builds to the GitHub Pages site. |
| `src/` · `tests/` | Language-appropriate source and test layout, pre-populated with a working smoke test. |
| `.github/` or `.azure-devops/` | CI workflow, PR template, issue templates, `CODEOWNERS`. |
| `docs/adr/0001-record-decisions.md` | An Architecture Decision Record seed — decisions are captured in-repo from day one. |
| `.gitignore` · `.editor-config` | Language-correct ignores and shared editor settings. |
| `CONTRIBUTING.md` · `SECURITY.md` · `LICENSE/NOTICE` | Contribution rules, vulnerability-reporting policy, and licensing appropriate to the project's context. |
| linter/formatter config | e.g. `ruff` + `black`, `clippy` + `rustfmt`, ESLint+Prettier — wired into CI so style is enforced, not debated. |

## 7.2 The branching model

Keel standardizes on a three-branch model — `main`, `dev`, `staging` — that maps cleanly to environments and is simple enough for non-specialist teams. Work happens on short-lived feature branches off `dev`; changes promote forward through `staging` to `main`, which is always releasable and always protected.

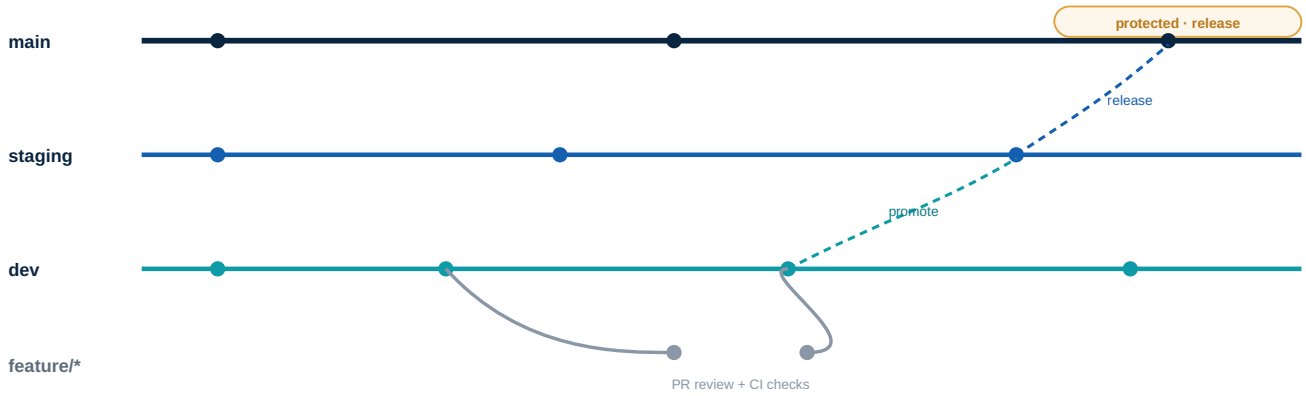


**Figure 4 — Branching model.** Feature branches off `dev` merge back via reviewed, CI-gated pull requests. Changes promote `dev` → `staging` → `main`. `main` is protected and always releasable. Keel applies the branches and protection automatically at initialization.

BRANCH ROLES AND DEFAULT PROTECTION

| Branch | Maps to | Default policy applied by Keel |
|---|---|---|
| `main` | Production / release | Protected: no direct pushes, PR + review required, CODEOWNERS approval, required CI checks green, linear history. |
| `staging` | Pre-production | PR required; required checks; the integration target for release candidates. |
| `dev` | Integration | Default working branch; CI runs on every push and PR. |
| `feature/*` | In-progress work | Short-lived; merged to `dev` via PR. Naming and PR template standardized. |

## 7.3 Early CI: green from birth

Every blueprint seeds a working CI pipeline so the very first run is meaningful. The MVP scope is deliberately modest — *build/compile, lint, and test* — but present and passing from commit one, which is the difference between a pipeline teams build on and a `TODO` they never get to. The same logical pipeline is expressed for GitHub Actions and (fast-follow) Azure Pipelines.

SEEDED CI BY LANGUAGE (MVP SCOPE)

| Language | Compile / build | Lint / format | Test |
|---|---|---|---|
| **Python** | install + import check | `ruff`, `black --check`, `mypy` | `pytest` smoke test |
| **Rust** | `cargo build` | `cargo clippy`, `cargo fmt --check` | `cargo test` |
| **TypeScript** | `tsc` / build | ESLint, Prettier check | `vitest` smoke test |

> **CI is itself a standard**
>
> Because the pipeline is part of the blueprint, improving it — adding a security scan, a coverage gate, an SBOM step — is a change to one file that every *future* project inherits immediately, and (via §11) can be offered to existing projects as an automated pull request.

# 8. Documentation as a first-class citizen

Docs-as-code, a standard structure, and a published site per project — from day one

Documentation is where standards usually go to die: created late, stored inconsistently, and never found when needed. Keel treats documentation as a born artifact of the project, not an afterthought. Every blueprint ships a `docs/` folder, a documentation build in CI, and a published **GitHub Pages** site — so the project documents itself from its first commit and the docs live in version control next to the code that makes them true.

## 8.1 Docs-as-code on GitHub Pages

The model is standard docs-as-code: Markdown under `docs/` is the source; a CI job builds it into a static site; GitHub Pages serves it. We recommend **Material for MkDocs** as the default generator — it is mature, Markdown-native, has excellent built-in search, deploys cleanly to Pages, and suits a polyglot audience. For Rust-heavy teams, **mdBook** (the tool behind the official Rust documentation) is offered as an alternative, since it is the natural fit for that ecosystem. The generator is a blueprint choice, not a per-developer decision.

## 8.2 A standard documentation skeleton

The most valuable thing Keel standardizes here is not the tooling but the *structure* — "clear standards so users know where to do what and how," in the words of this project's brief. Every site ships the same skeleton, pre-filled with the project's details and with prompts where human input is needed:

STANDARD DOCUMENTATION SKELETON (EVERY PROJECT)

| Page | What lives here |
|---|---|
| **Overview** | What the project does, who owns it, its status and links. Generated from initialization inputs. |
| **Getting started** | Clone, install, run, test — the path from zero to running locally. Verified against the actual scaffold. |
| **Architecture & decisions** | System overview plus Architecture Decision Records (ADRs). The "why," captured as it happens. |
| **Operations / runbook** | How to deploy, monitor, and respond when it breaks. Seeded with the standard headings. |
| **Reference / API** | Generated or hand-written interface docs, depending on language. |
| **Contributing** | Branching, PR, and review conventions — the same across every project, so contribution is frictionless. |

> **The compounding benefit**
>
> When every project documents itself in the same shape, knowledge becomes portable. An engineer joining their fifth Ramboll project already knows where "getting started" and the runbook live. Standardized structure turns documentation from a per-project lottery into an organizational asset.

## 8.3 Keeping docs honest

Because the docs build is part of CI, a broken docs build fails the pipeline — documentation cannot silently rot into non-building Markdown. And because the skeleton is part of the blueprint, improvements to the documentation standard propagate to new projects automatically and to existing ones via the upgrade mechanism described in §11.

# 9. Identity, access, and the SSO hub

One corporate sign-in for developers; least-privilege machine identity for automation

Keel handles two distinct identity problems: authenticating the *human* who starts a project, and acting as a *machine* against GitHub and Azure DevOps with the narrowest possible privileges. Keeping these separate is a core security principle.

## 9.1 Human identity: Entra ID single sign-on

Developers sign in to the Hub through Microsoft **Entra ID** using OpenID Connect — the authorization-code flow with PKCE — which fits Ramboll's Microsoft-aligned environment and means no Keel-specific passwords ever exist. In Rust this is well-trodden: the `openidconnect` crate provides the strongly-typed OIDC implementation, and `axum-oidc` wraps it for the Axum web layer, handling login redirects, token exchange, and session extraction. Entra publishes its OIDC discovery document at a well-known endpoint, so configuration is a single issuer URL plus the app registration.

## 9.2 Authorization: groups to roles

The ID token's group and role claims drive Keel's authorization. Entra security groups map to Keel roles and to the owning teams a developer may initialize projects for — so the form's "owning team" options are exactly the teams the signed-in user belongs to, and platform-team-only actions (publishing or editing blueprints) are gated by group membership.

INDICATIVE ROLE MODEL

| Role | Can |
|---|---|
| **Developer** | Initialize projects for their own teams from published blueprints; view the catalog. |
| **Team lead** | As Developer, plus manage their team's projects and approve team-scoped options. |
| **Platform engineer** | Author, version, and publish blueprints; manage golden-path policy; view audit. |
| **Auditor** | Read-only access to the catalog and the full audit log. |

## 9.3 Machine identity: least-privilege automation

Keel never uses a human's credentials to act on systems. Against GitHub it authenticates as a **GitHub App**, exchanging its private key for short-lived *installation access tokens* scoped to the specific organization and permissions it needs (repository administration, contents, Pages). octocrab implements this directly: build an app client from the App ID and RSA key, then call `.installation(id)` to obtain the installation-scoped client. Tokens are short-lived and cached only briefly, minimizing the blast radius of any compromise. Against Azure DevOps, an equivalent service principal (or scoped token) provides the parallel least-privilege identity.

> **Security posture, stated plainly**
>
> No standing broad-scope credentials; short-lived tokens minted per operation; the App's permissions are the ceiling on what Keel can ever do; secrets (the App private key, client secrets) live in a managed secret store such as Azure Key Vault, never in code or config; and every privileged action is audited. A full security review is a named follow-up before production (see §13).

# 10. Governance, standards-as-code, and drift

How a standard is set, evolved, and kept true over a project's life

Initialization sets a project on the golden path; governance keeps it there. This section describes how Keel makes standards durable rather than a one-time stamp — the bridge between the MVP and the frontier.

## 10.1 The blueprint as the unit of governance

Because every standard is encoded in a versioned, owned, tested blueprint, governance becomes a normal software-engineering activity. Changing the golden path — a new required CI check, a revised folder convention, an updated security default — is a pull request to a blueprint, reviewed by its owners and validated by the blueprint's own test that proves it still yields a green project. There is no separate "standards document" to drift out of sync with reality; the blueprint *is* the standard, and it is executable.

## 10.2 The catalog as system of record

Every project Keel creates is recorded in the software catalog with its owner, language, repository, documentation URL, and — importantly — the *blueprint version it was born from*. This single fact unlocks a great deal: leadership can answer "what do we have and who owns it?"; security can answer "which projects predate the standard that added secret scanning?"; and the platform team can measure how much of the estate is on the current golden path. The catalog emits portable metadata so a future portal (Backstage, Port) can consume it without rework.

## 10.3 Drift: the gap between born-compliant and stays-compliant

A project initialized perfectly in Q4 will, by Q2, no longer match a golden path that has since improved — and may have been hand-edited away from standard. This gap is **drift**, and naming it is the first step to managing it. Keel's catalog knows each project's blueprint version; comparing that against the current version identifies projects that are behind, and comparing actual repository settings against expected settings identifies projects that have diverged. In the MVP this is *reported* (a project's place on the path is visible); the frontier section describes *remediating* it automatically.

## 10.4 Scorecards and the tech radar

Borrowing the now-standard scorecard pattern (popularized by OpsLevel and Cortex), Keel can score each project against the golden path across dimensions such as reliability, security, documentation, and operational readiness — turning "are we on standard?" into a visible, trackable metric rather than an opinion. A lightweight tech radar records which languages and tools are blessed (and which are deprecated), so the blueprint catalog and the organization's stated direction stay aligned. These are introduced conceptually here and scheduled in §12; they are explicitly *not* in the Q4 MVP.

# 11. Frontier capabilities: the bold bets

Where this goes once the foundation is solid — kept out of the MVP on purpose

The brief asked us to push frontier ideas while keeping the MVP simple. This section does exactly that: it sketches the ambitious capabilities that turn Keel from a one-time scaffolder into a living platform, and §12 then walls them off from the Q4 delivery. Each is designed as a natural extension of the same engine — blueprints, catalog, and workflow — not a separate product.

## 11.1 AI-assisted scaffolding

The single most natural frontier for Keel. Rather than navigating a catalog and a form, a developer describes intent in natural language — "a Python service that ingests sensor data and exposes a REST API" — and an assistant proposes the blueprint, pre-fills parameters, and explains its choices for confirmation. Beyond selection, generative models can draft the *content* that humans usually skip: a real first README, a starter ADR capturing the initial design, runbook stubs tailored to the service, and example tests. Crucially, AI operates *within* the golden path — it fills in a standardized, validated structure rather than free-styling — so its output is bounded by the blueprint. Industry sentiment is striking here: a large majority of practitioners now see AI as critical to the future of platform engineering, making this a strategic rather than speculative bet.

## 11.2 Self-updating blueprints (golden-path upgrades as pull requests)

The answer to drift. When a blueprint improves, Keel can generate pull requests against *existing* repositories that were created from earlier versions — re-rendering the changed files and opening a reviewed PR, in the spirit of automated dependency tools like Renovate but for the golden path itself. A security default added once propagates across the estate as a wave of small, reviewable PRs rather than a migration project. This is the capability that makes "day-2 matters as much as day-0" real.

## 11.3 Continuous drift detection and auto-remediation

Building on §10.3, Keel continuously compares live repository configuration (branch protection, required checks, settings) against the golden-path expectation and either flags divergence on a scorecard or opens a remediation PR/automated fix. Standards become self-healing instead of slowly eroding.

## 11.4 Paved-road scorecards and compliance gates

Scorecards (introduced in §10.4) graduate from visibility to *incentive*: surfaced in the catalog, optionally gating promotions (e.g. a project must meet a documentation and security bar before it can be released from `staging` to `main`). The golden path becomes measurable and, where appropriate, enforced.

## 11.5 Policy-as-code guardrails

Organizational policy — naming, licensing, mandatory checks, data-handling rules — is expressed as code (e.g. OPA/Conftest-style policies) and evaluated both at initialization (you cannot create a non-compliant project) and continuously in CI. Guardrails replace gatekeepers: developers move fast inside boundaries the platform guarantees.

## 11.6 Telemetry-driven golden paths

Keel measures what developers actually do — which blueprints are chosen, where initialization fails, which generated files are immediately deleted or heavily edited — and feeds that back into blueprint design. The golden path stops being a guess and becomes an evidence-based product that improves with use.

### FRONTIER CAPABILITIES MAPPED TO HORIZON

| Capability | Horizon | MVP posture |
|---|---|---|
| AI-assisted scaffolding | LATER | Spike only; design the engine so blueprints are AI-fillable. |
| Self-updating blueprint PRs | LATER | Out of MVP; catalog records blueprint version to enable it. |
| Drift detection & remediation | LATER | MVP records version; reporting in H1; remediation in H2. |
| Scorecards & compliance gates | LATER | Not in MVP; conceptually reserved in the catalog model. |
| Policy-as-code guardrails | LATER | MVP enforces a few rules inline; generalize later. |
| Telemetry-driven paths | LATER | MVP captures basic events; analysis later. |

**Design for the frontier, build the foundation**

The discipline this paper recommends: make every MVP decision compatible with these bets — version blueprints, record provenance in the catalog, keep the engine deterministic and event-emitting — but ship none of them in Q4. The foundation must be solid before the ship is built on it.

# 12. MVP scope for Q4 and the roadmap

Deliberately small, deliberately solid, deliberately extensible

The Q4 objective is a real, usable initialization platform that does a narrow thing excellently: sign in, pick one of three blueprints, and get a green, documented, cataloged GitHub repository. Everything in §11 is explicitly excluded to protect that delivery.

## 12.1 In and out of scope for Q4

Q4 MVP SCOPE

| In scope | Out of scope (deferred) |
|---|---|
| SSO hub (Entra ID OIDC) with role mapping | AI-assisted scaffolding (spike only) |
| 3 blueprints: Python, Rust, TypeScript | Azure DevOps parity (GitHub-first; ADO fast-follow) |
| Repo creation + structured initial commit | Self-updating blueprint PRs / auto-remediation |
| Branches main/dev/staging + protection | Scorecards & compliance gates |
| Seeded CI: build + lint + test | Generalized policy-as-code engine |
| GitHub Pages docs site + standard skeleton | CLI (web Hub only for MVP) |
| Catalog entry + audit log + blueprint versioning | Preview environments, telemetry analytics |

## 12.2 Roadmap

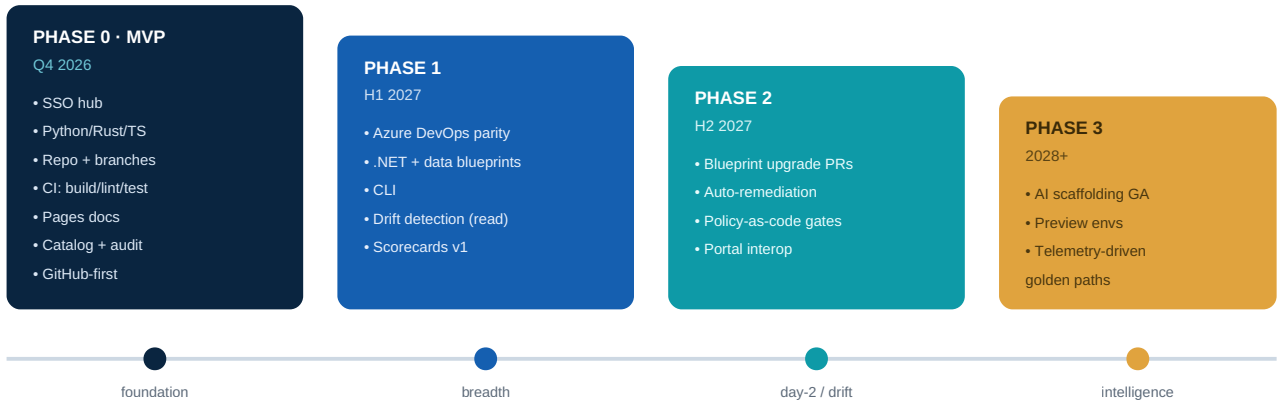


**Figure 5 — Phased roadmap.** Phase 0 ships a solid foundation; later phases add breadth (more targets and blueprints), day-2 governance (drift and upgrades), and intelligence (AI, telemetry). Each phase is independently valuable.

## 12.3 Success metrics

We propose measuring Keel against outcomes, not output. Baselines should be captured before launch so improvement is provable.

PROPOSED SUCCESS METRICS

| Metric | Why it matters |
|---|---|
| **Time-to-first-commit** | The headline efficiency gain: days of setup → minutes. The most tangible developer win. |
| **% new projects via Keel** | Adoption. If the golden path is truly easiest, this trends toward the large majority of new work. |
| **% estate on current golden path** | Consistency and drift — the standards story, made measurable via the catalog. |
| **Documentation coverage** | Share of projects with a building, published docs site (≈100% by construction for Keel-born projects). |
| **First-run CI pass rate** | Quality of the scaffold: every Keel project should be green on commit one. |
| **Developer satisfaction** | Does the path feel like a concierge or a constraint? Surveyed periodically. |

# 13. Risks and mitigations

The honest list — what could go wrong and how we contain it

A whitepaper that names no risks is marketing. These are the failure modes we consider most material, with the mitigation built into the approach.

KEY RISKS AND MITIGATIONS

| Risk | Sev. | Mitigation |
|---|---|---|
| **Scope creep** — the frontier (§11) leaks into Q4 and the MVP slips. | High | Hard scope wall in §12; frontier items explicitly out; ruthless "is this needed to create one green repo?" test for every feature. |
| **Blueprint maintenance burden** — blueprints rot as languages/tools move. | High | Blueprints are tested in CI (initialized into throwaway repos on every change); clear platform-team ownership; small MVP catalog (3) keeps the surface manageable. |
| **Adoption / "golden cage"** — teams route around Keel if it feels restrictive. | High | Make the path genuinely easiest, not mandatory; provide escape hatches; co-design blueprints with pilot teams; measure satisfaction and divergence and respond. |
| **Two-VCS complexity** — GitHub + Azure DevOps doubles integration effort. | Med | GitHub-first in MVP; abstract a `VcsProvider` trait so Azure DevOps is an additive implementation, not a rewrite. |
| **Security of automation identity** — a powerful App is a target. | High | Least-privilege GitHub App; short-lived installation tokens; secrets in Key Vault; full audit; dedicated security review before production (named follow-up). |
| **Rust talent scarcity** — fewer engineers, slower ramp. | Med | Small, well-bounded MVP; mainstream crates not exotic ones; invest in internal Rust enablement (Keel itself becomes a flagship Rust project and recruiting signal). |
| **Over-standardization** — one path cannot fit genuinely different needs. | Med | Multiple blueprints per intent over time; parameters for legitimate variation; treat frequent escape-hatch use as a signal to evolve the path, not police it. |
| **External API change/limits** — GitHub/Azure/Entra APIs shift or throttle. | Low | Mature, supported crates; idempotent, retrying workflow; rate-limit awareness; pinned, tested dependency versions. |

# 14. Recommendation and next steps

What we are asking for, and what happens next

We recommend proceeding with **Keel** as the project-initialization layer of the Ramboll Developer Platform, targeting the Q4 MVP defined in §12: a Rust-native, SSO-fronted hub that initializes standards-compliant GitHub repositories from three language blueprints, complete with the `main` / `dev` / `staging` branching model, a green CI pipeline, and a published GitHub Pages documentation site. The scope is small by design and the architecture is chosen so that the ambitious capabilities in §11 are later additions to the same engine, not future rewrites.

The strategic argument is simple: the cheapest, most durable place to apply an engineering standard is at the moment a project is created. Keel converts Ramboll's standards from prose nobody reads into executable blueprints applied automatically — buying consistency across a polyglot, distributed estate and giving every team back the days they currently spend on undifferentiated setup.

**Immediate next steps**

1. **Endorse scope & codename.** Confirm the Q4 MVP boundary in §12 and a working name for the platform.
2. **Stand up a small team.** A lead plus two engineers with (or building) Rust capability; identify the platform-team owners of the blueprints.
3. **Select 2–3 pilot teams.** Co-design the first blueprints with real teams across the Python, Rust, and TypeScript paths.
4. **Commission the follow-ups this paper deferred:** a security review of the automation identity, a deployment-topology and hosting decision, and a light cost model.
5. **Provision the foundations.** Register the GitHub App and the Entra ID app registration; agree the target GitHub org(s) and Azure DevOps project(s).
6. **Define baselines.** Capture today's time-to-first-commit and consistency baseline so the MVP's impact is measurable.

Lay the keel first. Everything else — the breadth, the day-2 governance, the intelligence — is built upon it.

# A. Proposed technology stack and crates

Indicative; all are mature, mainstream crates. Versions to be pinned at build time.

RUST CRATE STACK

| Crate | Role in Keel | Notes |
|---|---|---|
| `axum` + `tower` | HTTP API and Hub backend | Async web framework on the tokio stack; middleware for auth, rate-limiting, tracing. |
| `tokio` | Async runtime | Concurrency for parallel initializations and fan-out API calls. |
| `octocrab` | GitHub integration | Typed API for repos/contents/refs/branches; GitHub App + installation-token auth; lower-level `post` / `put` for endpoints not yet typed (org repo creation, branch protection). |
| `azure_devops_rust_api` | Azure DevOps integration (Phase 1) | Microsoft-maintained, generated from the Azure DevOps REST 7.1 spec; per-area opt-in features (Git, Build, Core). Requires Rust ≥ 1.80. |
| `openidconnect` + `axum-oidc` | Entra ID single sign-on | Strongly-typed OIDC; authorization-code + PKCE; `axum-oidc` handles login/session for the Hub. |
| `jsonwebtoken` | GitHub App JWT | Signs the App JWT from the RSA private key (used by octocrab's app auth). |
| `minijinja` | Blueprint rendering | Jinja2-compatible, minimal-dependency template engine; renders file contents and paths. `tera` is an alternative. |
| `serde` (+ `serde_yaml`, `serde_json`) | Manifests & config | Parses `blueprint.yaml`, request bodies, and emitted catalog metadata. |
| `garde` / `validator` | Parameter validation | Validates developer inputs against manifest rules before any side effect. |
| `sqlx` + PostgreSQL | Catalog & audit store | Compile-time-checked queries; append-only audit; catalog of projects + blueprint provenance. |
| `tracing` + `tracing-subscriber` | Observability | Structured logs and spans across the initialization workflow. |

> **On `octocrab` coverage**
>
> octocrab provides a high-level typed API *and* a documented lower-level HTTP interface that reuses the same authentication and configuration. Where the typed surface does not yet cover an endpoint (notably organization repository creation and branch-protection rules), Keel calls the lower-level `post`/`put` methods — a supported, first-class pattern, not a workaround.

# B. Example blueprint manifest

A fuller `blueprint.yaml` illustrating parameters, conditionals, and post-actions

```yaml
apiVersion: keel/v1
kind: Blueprint
metadata:
  name: python-service
  title: "Python Service"
  description: "REST or worker service — the Ramboll Python golden path."
  version: "1.4.0"
  owner: platform-engineering
  tags: [python, service, golden-path]

parameters:
  - id: project_name
    title: "Project name"
    type: string
    pattern: "^[a-z][a-z0-9-]{2,40}$"
    required: true
  - id: owning_team
    title: "Owning team"
    type: enum
    source: entra-groups          # constrained to the user's teams
    required: true
  - id: service_kind
    type: enum
    values: [rest-api, worker]
    default: rest-api
  - id: target
    type: enum
    values: [github, azure-devops]
    default: github

template:
  root: template/
  include:
    - "**/*"
  conditions:                       # render some files only for a choice
    - when: "service_kind == 'rest-api'"
      paths: [ "src/{{ project_name }}/api.py" ]

repository:
  visibility: internal
  default_branch: main
  branches: [main, dev, staging]
  protect:
    - branch: main
      require_pull_request: true
      required_reviews: 1
      require_codeowners: true
      required_checks: [ci]

postActions:
  - create_repository
  - commit_template
  - setup_branches
  - apply_protection
  - enable_ci
```

```
  - publish_docs            # MkDocs -> GitHub Pages
  - register_in_catalog      # records blueprint version 1.4.0
```

The manifest is declarative: the Hub builds the form from `parameters`, the engine renders `template/` (honoring `conditions`), and the workflow runs `postActions` in order. Recording the blueprint version in the catalog is what later enables drift detection and self-updating PRs (§11).

# C. Glossary

Shared vocabulary for this paper

| Term | Meaning |
|---|---|
| **IDP** | Internal Developer Platform — the self-service layer that lets engineers build and run software via paved roads and guardrails. Keel is its initialization layer. |
| **Golden path** | An opinionated, well-documented, fully-supported way to build a kind of software — the route the platform makes easiest. |
| **Blueprint** | Keel's unit of standardization: a manifest + template tree + post-actions that produce a standards-compliant project. |
| **Scaffolder** | The engine that turns a blueprint and a developer's inputs into a created, configured project (cf. Backstage Software Templates). |
| **Drift** | The growing gap between a project as initialized and the current golden path (or its own intended configuration). |
| **Scorecard** | A score of a project against standards (reliability, security, docs, readiness) used to make compliance visible. |
| **Docs-as-code** | Treating documentation like code: Markdown in the repo, built in CI, published as a site. |
| **OIDC** | OpenID Connect — the identity protocol used for single sign-on via Entra ID. |
| **GitHub App / installation token** | A machine identity that mints short-lived, narrowly-scoped tokens — how Keel acts on GitHub with least privilege. |
| **ADR** | Architecture Decision Record — a short in-repo note capturing a significant decision and its rationale. |
| **Control plane** | A long-lived service that orchestrates and configures other systems — what Keel is. |
| **Idempotent** | An operation that can be repeated without changing the result beyond the first application — key to safe retries. |

END OF DOCUMENT